面向对象程序设计
复习

李际军　lijijun@cs.zju.edu.cn

第一部分

判断题

1.Default constructor must be defined, if parameterized constructor is defined and the object is to be created without arguments. ( )

2. The destructor can be called before the constructor if required.( )

3. In C++, overloading a function and overriding a function are essentially the same thing. ( )

4. In C++, overloading a function and overriding a function are essentially different. ( )

5. Passing an object using copy constructor and passing by value are the same. ( )

6. Copy constructors are overloaded constructors.( )

7. If a class is enclosing more than one class, than it can be called as base class of those classes. ( )

8. If a base class is added with a few new members, its subclass must also be modified. ( ) (TFFTFTFF)

9. In C++, using the delete[] operator on a pointer allocated with new[] will only call the destructor for the first element in the allocated array. ( )

10. In C++, using the delete[] operator on a pointer allocated with new[] will automatically call the destructor for each element in the allocated array.( )

11. If one class have derived the base class privately, then another class can't derive the base class publicly.( )

12. All the derived classes can access only a few members of base class that other derived classes can't access at the same time, in hierarchical inheritance. ( )

13. All the operators can be overloaded using the member function operator overloading.( )

14. A function can have both the const and non-const version in the same program. ( ) (FTFFFT)

15. A private function of a derived class can be accessed by the parent class.( )

16. A derived class object can access the public members of the base class no matter which type of inheritance it is. ( )

17. There can be a try block without catch block but vice versa is not possible.( )

18. If both base and derived classes can catch exceptions， then the catch block of the derived class should be defined before base class to catch exceptions. ( )

19. Passing by reference and passing by value can’t be done simultaneously in a single function argument list. ( )

20. If an object is passed by reference to a function then it must be returned by reference.( )

(FFTTFF)

单选题

1. Which of the following is correct about static polymorphism? (A)

A. In static polymorphism, the conflict between the function call is resolved during the compile time;
B. In static polymorphism, the conflict between the function call is resolved during the run time;
C. In static polymorphism, the conflict between the function call is never resolved during the execution of a program;
D. In static polymorphism, the conflict between the function call is resolved only if it required;

2. What is the difference between references and pointers? (A)

E. References are an alias for a variable whereas pointer stores the address of a variable
F. References and pointers are similar
G. References stores address of variables whereas pointer points to variables
H. Pointers are an alias for a variable whereas references stores the address of a variable

3. How constructors are different from other member functions of the class? (D)

I. Constructor has the same name as the class itself
J. Constructors do not return anything
K. Constructors are automatically called when an object is created
L. All of the mentioned

4. Which of the following operator cannot be used to overload when that function is declared as friend function? (D)

A. -=;
B. ||;
C. ==;
D. [];

5. Which rule is correct for the general function but will not affect the friend function? (A)

E. private and protected member can be accessed anywhere
F. private and protected members of a class cannot be accessed from the outside of the same class
G. protected member can be accessed anywhere
H. private member can be accessed anywhere

6. Which is the correct syntax for declaring the type of this in a member function?(D)

I. classType [cv-qualifier-list] *const this;
J. classType const[cv-qualifier-list] *this;
K. [cv-qualifier-list]*const classType this;
L. [cv-qualifier-list] classType *const this;

7. Which is the correct statement about pure virtual functions? (C)

A. They should be defined inside a base class||;
B. Pure keyword should be used to declare a pure virtual function
C. Pure virtual function is implemented in derived classes
D. Pure virtual function cannot implemented in derived classes

8. Abstract class A has 4 virtual functions. Abstract class B defines only 2 of those member functions as it extends class A. Class C extends class B and implements the other two member functions of class A. Choose the correct option below. (D)

E. Program won’t run as all the methods are not defined by B
F. Program won’t run as C is not inheriting A directly
G. Program won’t run as multiple inheritance is used
H. Program runs correctly

9. What will be the output of the following C++ code? (B)

```
#include<iostream>
using namespace std;
class Test {
private:
    static int count;
public:
    Test& fun();
};
int Test::count = 10;
Test& Test::fun() {
     cout << --Test::count << " ";
     return *this;
}
int main() {
    Test t;
     t.fun().fun().fun().fun();
      return 0;
}
```

A. 10 10 10 10;
B. 9 8 7 6
C. 7 7 7 7
D. 10 9 8 7

10. How many times does the following program call the copy constructor of class A ? (B)

```
# include <iostream>
using namespace std;
class A {
public:
    A() { cout << "A()" << endl; }
    A(const A&) { cout << "A(const &)" << endl; }
    ~A() { cout << "~A()"<<endl; }
};
void Func(A a) { A a2 = a; return; }
int main() {
    A a;
    for (int i = 0; i < 2; i++) {
    Func(a);
    }
}
```

A. 2;
B. 4
C. 8
D. 9

填空题

```
1.#include <iostream>
using namespace std;
void f3( ) {
    double a=0;
    try {throw a;}
    catch(double) {
        cout<<"OK3!"<<" "; throw;}
    cout<<"end3"<<" ";
}
void f2( ) {
   try { f3( ); }
   catch(int) {cout<<"Ok2!"<<" ";}
   cout<<"end2"<<" "; }
void f1( ) {
    try {f2( );}
    catch(char) { cout<<"OK1!"; }
    cout<<"end1"<<" ";
}
int main( ) {
    try {f1( );}
    catch(double) {cout<<"OK0!"<<" ";}
     cout<<"end0"<<" ";
return 0;
}
```

The result is:

OK3! OK0! end0

2. What will be the output of the following code?

```
#include <iostream>
using namespace std;
class A {
public:
A() { increase(); }
A(const A&) { increase(); }
static void reset() { data = 0; }
static void increase() { data++; }
static void print() { cout << data; }
private:
static int data;
};
int A::data = 1;
void foo(A a) {
a.print(); }
int main() {
A::print();
A a;
foo(a);
cout << endl;
a.print();
a.reset();
foo(a);
}
```

The result is:

13

31

3. What will be the output of the following code?

```
#include <iostream>
using namespace std;
class A {
public:
A() { cout << "A()" << endl; }
~A() { cout << "~A()" << endl; }
};
class B {
public:
B() { cout << "B()" << endl; throw 1; }
~B() { cout << "~B()" << endl; }
private:
A a;
};
int main() {
try { B b; }
catch (int) { cout << "exception
catched!" << endl; }
}
```

The result is:

A()

B()

~A()

exception catched!

4. What will be the output of the following code?

```
#include <iostream>
using namespace std;
template <typename T>
class FF {
T a1, a2, a3;
public:
FF(T b1, T b2, T b3) :a1(b1), a2(b2), a3(b3) { }
T Sum() const { return a1 + a2 + a3; }
};
template <>
class FF<double> {
double a1, a2, a3;
public:
FF(double b1, double b2, double b3) :
        a1(b1), a2(b2), a3(b3) {}
double Sum() const {
cout << "FF(double)" << endl; return a1 + a2 +
a3 + 10.0; }
};
int main() {
FF<double> d(10.0, 20.0, 10.0);
FF<float> f(10.0f, 20.0f, 10.0f);
cout << (int)d.Sum() << endl << (int)f.Sum() <<
endl;
FF<int> x(2, 3, 4), y(-2, -3, -4);
cout << x.Sum() << endl << y.Sum() << endl;
}
```

The result is:

FF(double)

50

40

9

-9

程序填空题

1. Fill-in-Blank - P - Polygon class

The following Polygon class represents a geometric polygon using a list of Point objects. It includes operator overloads for adding a point, comparing two polygons, and stream insertion for displaying the polygon's points. Please fill in the blanks of the following code to finish the implementation.

The code to complete:

```
#include <iostream>
#include <vector>
using namespace std;
class PolygonException {
private:
      string m_message;
public:
      PolygonException(const string& message) : m_message(message) {}
      const string& what() const {
     return m_message;     }
};
```

```
class Point {
public:
    float m_x, m_y;
    Point(float x = 0.0, float y = 0.0) :m_x(x), m_y(y)
    friend ostream& operator<<(ostream& os, const Point& p) {
        os << "(" << p.m_x << ", " << p.m_y << ")";
        return os;
    }
};
class Polygon {
private:
    vector<Point> m_points;
public:
    Polygon() {}
    Polygon& operator+=(const Point& p) {
        m_points.push_back(p);
        Return   *this___;
}
```

```
Bool  operator==_(const Polygon& other) const {
if (m_points.size() != other.m_points.size()) {
      throw   PolygonException("Cannot compare polygons of different sizes.");
}
for (size_t i = 0; i < m_points.size(); ++i) {
      if (m_points[i].m_x != other.m_points[i].m_x || m_points[i].m_y !=
other.m_points[i].m_y)
                  return false;
       }
       return true;
}
bool operator!=(const Polygon& other ) const {
return !(*this == other);
}
```

```
friend ostream& operator<<(ostream& os, const Polygon& poly) {
    os << "Polygon with " << poly.m_points.size() << " points: ";
    for (auto it = poly.m_points.begin(); it <  poly.m_points.end()  ; ++it) {
            os << *it << " ";
        }
        return os;
        }
        ~Polygon() {}
};

int main() {
      try {
            Polygon poly1;
            poly1 += Point(1, 2);
            poly1 += Point(3, 4);
            poly1 += Point(5, 6);
```

```
Polygon poly2;
poly2 += Point(1, 2);
poly2 += Point(5, 6);
poly2 += Point(7, 8);
Polygon poly3;
poly3 += Point(9, 10);
cout << poly1 << endl;
cout << poly2 << endl;
cout << poly3 << endl;

if (poly1 == poly2) {
cout << "Polygons are equal." << endl;        }
else {
cout << "Polygons are not equal." << endl;        }
if (poly2 == poly3) {
cout << "Polygons are equal." << endl;        }
else {
cout << "Polygons are not equal." << endl; }    }
```

```
catch (const    PolygonException& e  {
        cout << "Polygon error: " << e.what() << endl;     }

cout << "End of test." << endl;
return 0;
}
```

The desirable output:

```
Polygon with 3 points: (1, 2) (3, 4) (5, 6)
Polygon with 3 points: (1, 2) (5, 6) (7, 8)
Polygon with 1 points: (9, 10)
Polygons are not equal.
Polygon error: Cannot compare polygons of different sizes. End of test.
```

2. Fill-in-Blank - P - library

The following C++ program demonstrates an object-oriented approach to managing a library system that can handle both books and multimedia items.Please fill in the blanks of the following code to finish the implementation.

The code to complete:

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;
class Book {
protected:
    string title;
    string author;
public:
     Book(const string& title, const string& author) : title(title), author(author) {}
     virtual ~Book() {}
     virtual  void display() const {
        cout << "Book - Title: " << title << ", Author: " << author << endl;     }
};
```

```
class Novel : public Book {
public:
    Novel(const string& title, const string& author) : Book(title, author) {}
};
class Magazine : public Book {
private:
    int issueNumber;
    string publicationDate;
public:
    Magazine(const string& title, const string& author, int issueNumber, const
string& publicationDate)        : Book(title, author), issueNumber(issueNumber),
publicationDate(publicationDate) {}
    void display() const override {
        cout << "Magazine - Title: " << title << ", Author: " << author          << ",
Issue: " << issueNumber << ", Date: " << publicationDate << endl;
    }
};
```

```
class Multimedia {
protected:
        string title;
        string creator;
public:
      Multimedia(const string& title, const string& creator) : title(title), creator(creator)
{}
      virtual ~Multimedia() {}
      virtual void display() const {
cout << "Multimedia - Title: " << title << ", Creator: " << creator << endl;
     }
};
class Movie : public Multimedia {
public:
     Movie(const string& title, const string& director) : Multimedia(title, director) {}
};
```

```
class MusicAlbum : public Multimedia {
private:
    int releaseYear;
    string genre;
public:
    MusicAlbum(const string& title, const string& artist, int releaseYear, const string&
genre)  :  Multimedia(title, artist)  , releaseYear(releaseYear), genre(genre) {}
void display() const  override {
        cout << "Music Album - Title: " << title << ", Artist: " << creator            << ",
Year: " << releaseYear << ", Genre: " << genre << endl;
    }
};
template <class T>
class Library {
private:
    vector<T*> items;
    size_t capacity;
public:
```

```
Library(size_t cap) : capacity(cap) {}
~Library() {
    cout << "Deleting items" <<endl;
    for (T* item : items) {          delete item;        }
}
bool addItem(   T* item   )  {
     if (items.size() >= capacity) {
          return false;
      }
      items.push_back(item);
      return true;
}
void displayItems() const {
       for (const T* item : items) {
            item->display()    ;
          }
    }
};
```

```
typedef Library<Multimedia> MultimediaLibrary;
int main() {
    {
     Library<Book> bookLibrary(5);
     bookLibrary.addItem(new Novel("1984", "George Orwell"));
     bookLibrary.addItem(new Magazine("National Geographic", "Various", 120, "July
2021"));
     cout << "Book Library:" << endl;
     bookLibrary.displayItems();
    }
    MultimediaLibrary multimediaLibrary(5);
    multimediaLibrary.addItem(new Movie("Inception", "Christopher Nolan"));
    multimediaLibrary.addItem(new MusicAlbum("Abbey Road", "The Beatles", 1969,
"Rock"));
    cout << "Multimedia Library:" << endl;
    multimediaLibrary.displayItems();
    return 0;
}
```

The desirable output:
Book Library:
Book - Title: 1984, Author: George Orwell
Magazine - Title: National Geographic, Author: Various, Issue: 120, Date: July 2021
Deleting items
Multimedia Library:
Multimedia - Title: Inception, Creator: Christopher Nolan
Music Album - Title: Abbey Road, Artist: The Beatles, Year: 1969, Genre: Rock
Deleting items

主观题

1. Vehicle Construction Simulation

We aim to design a simple vehicle construction simulation system that constructs different types of vehicles. The system follows the Builder design pattern which allows for the creation of complex objects step by step.
There are 7 classes in total in the system:
Vehicle: A class representing a generic vehicle, with some common attributes.
Car: A class representing a car, derived from Vehicle, with a specific attribute like carEngine .The engine can be an electric one or a gasoline one.
Truck: A class representing a truck, derived from Vehicle, with a specific attribute like cargoCapacity.
VehicleBuilder: An abstract class representing the steps to build a vehicle.
CarBuilder: A concrete builder class that implements VehicleBuilder for building a car.
TruckBuilder: A concrete builder class that implements VehicleBuilder for building a truck.
Director: A class that is responsible for executing the building tasks.

The implementation for Vehicle, Car,and TruckBuilder classes are as follows:

```
// Abstract base class for vehicles
class Vehicle {
protected:
	string m_bodies;
	string m_tires;
public:
	virtual void display() const = 0;
	void setBodies(const string& body) { m_bodies = body; }
	void setTires(const string& type) { m_tires = type; }
	virtual ~Vehicle() {}
};
// Concrete class for cars
class Car : public Vehicle {
private:
	string m_carEngine;
public:
	void setEngine(const string& engine) { m_carEngine = engine; }
	void display() const override {
		cout << "Car configuration: " << endl <<	"engine: " << m_carEngine << ", bodies: " <<
m_bodies << ", tires: " << m_tires << endl;
	}
 };
```

```
// Concrete builder for trucks
class TruckBuilder : public VehicleBuilder {
private:
    float m_capacity;
public:
    TruckBuilder(float cargoCapacity) : m_capacity(cargoCapacity) {}
    Vehicle* buildVehicle() override {
        Truck* truck = new Truck();
        truck->setTires("Truck Tires");
        truck->setBodies("Truck Bodies");
        truck->setCargoCapacity(m_capacity);
        return truck;
    }
};
```

Your task is to implement Truck,VehicleBuilder, CarBuilder, Director classes such that the following main function works properly.

```
#include <iostream>
#include <string>
using namespace std;
int main() {
	Director director;
	CarBuilder carBuilder("Electric Engine");
	TruckBuilder truckBuilder(10.0f);
// Construct a car with an electric engine
	Vehicle* car = director.construct(carBuilder);
	car->display();
// Construct a truck with 10 tons cargo capacity
	Vehicle* truck = director.construct(truckBuilder);
	truck->display();
	delete car;
	delete truck;
	return 0;
}
```

The output of the program will be:

Car configuration:
engine: Electric Engine, bodies: Car Bodies, tires: Car Tires
Truck configuration:
cargo capacity: 10 tons, bodies: Truck Bodies, tires: Truck Tires

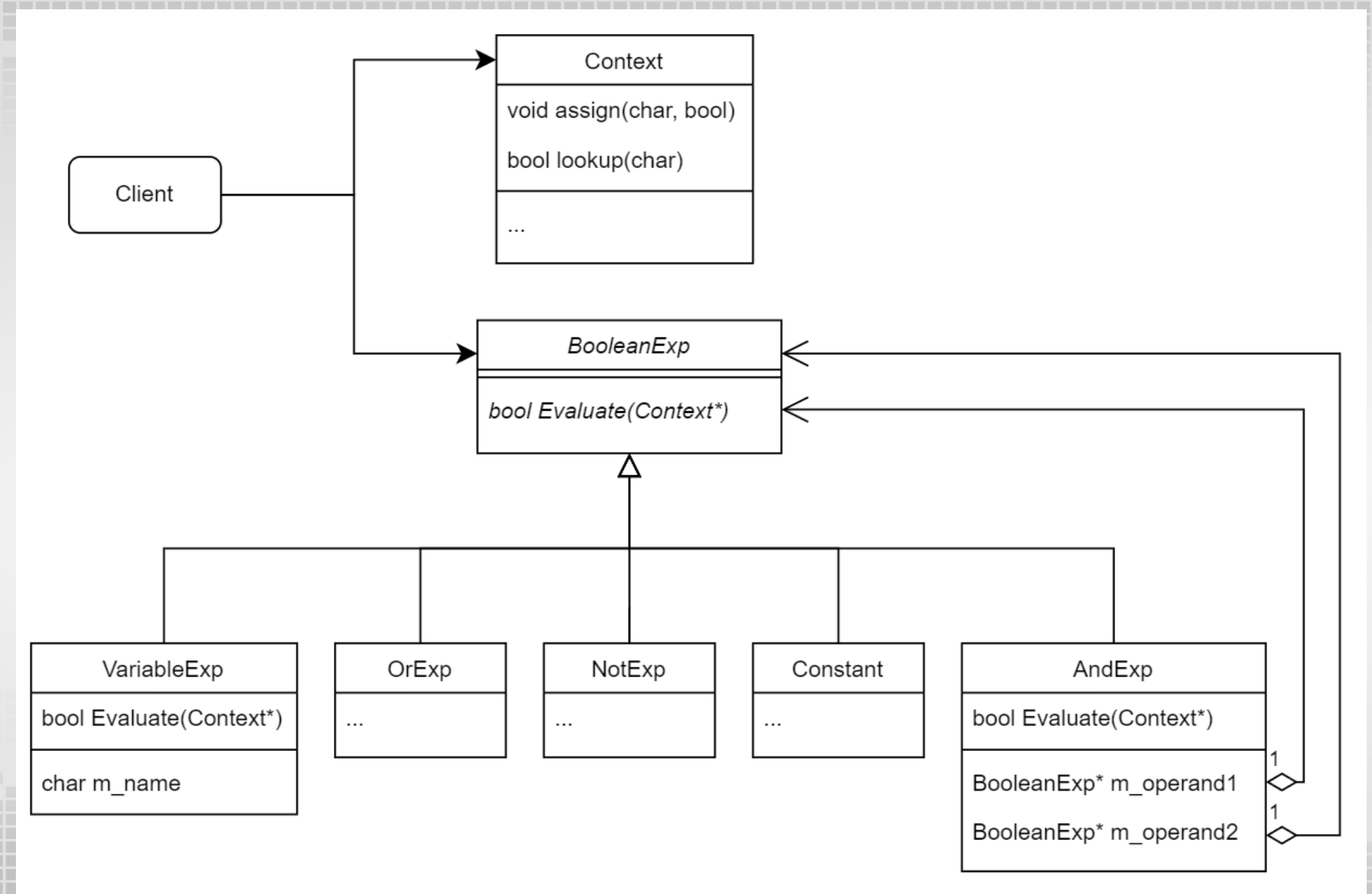
2. The Representation and Evaluation of Boolean Expressions

In computer science, we can choose to generate Boolean expressions via production rules as follows:
BooleanExp := VariableExp | Constant | OrExp | AndExp | NotExp
AndExp := BooleanExp and BooleanExp
OrExp := BooleanExp or BooleanExp
NotExp := not BooleanExp
Constant := true | false VariableExp := A | B | C | … | X | Y | Z
Each production rule has a *head*, or *nonterminal*, symbol before :=, The symbol | in the production rule indicates that this rule is the combination of several production rules. For instance, the first production rule implies: BooleanExp := VariableExp, BooleanExp := Constant, BooleanExp := OrExp, etc. The head or nonterminal symbols can appear in the right part of any production rule, which means the right part of the production rule for that head symbol can be substituted to replace that head symbol. We can also define the terminal symbols as those symbols that do not appear as head symbols in any production rule. In this example, they are and,or,not,true,false, A | B | C | … | X | Y | Z.

Now we give an example to generate a Boolean expression "X and true"using these production rules with the following 4 iterative steps:

1. Substitute the right part of BooleanExp := AndExp into the result of step 1, the resulted expression is AndExp;
2. Substitute the right part of AndExp := BooleanExp and BooleanExp into the result of step 2, the result is BooleanExp and BooleanExp
3. Substitute the BooleanExp := VariableExp and BooleanExp := Constant into the result of step 3, we get the expression VariableExp and Constant
4. Substitute VariableExp := X and Constant:= true into the result of step 4, we get the final expression: X and true

We can design a hierarchy of C++ classes to represent these production rules as follows:



The code for BooleanExp, AndExp, VariableExp classes are as follows:

```
class BooleanExp {
public:
    virtual ~BooleanExp() {}
    virtual bool Evaluate(Context *) = 0;
};
class AndExp : public BooleanExp {
public:
    AndExp(BooleanExp *operand1, BooleanExp *operand2) :
m_operand1(operand1), m_operand2(operand2) {}
    virtual ~AndExp() { delete m_operand1; delete m_operand2; }
    virtual bool Evaluate(Context *pContext) {
        return m_operand1->Evaluate(pContext) && m_operand2-
>Evaluate(pContext);
    }
private:
    BooleanExp *m_operand1, *m_operand2;
};
```

The code for BooleanExp, AndExp, VariableExp classes are as follows:

```
class VariableExp : public BooleanExp {
public:
    VariableExp(const char variableName) : m_name(variableName) {}
    virtual ~VariableExp() {}
    virtual bool Evaluate(Context *pContext) {
        return pContext->lookup(m_name);
    }
private:
    char m_name;
};
```

The Context class provides interfaces to store variable names and its assigned boolean values:

```
class Context {
public:
    void assign(char name, bool value);
    bool lookup(char name);
};
```

Your Task is to implement OrExp, NotExp, Constant classes, and complete the Context class such that the following main function works properly.

```
#include <iostream>
using namespace std;
int main() {
// set the flag: print bool values as "true" and "false", not "1" and "0"
      cout << boolalpha;
// represent the boolean expression "(Y and true) or (X and (not Y))"
      AndExp *pAndExp1 = new AndExp(new VariableExp('Y'), new Constant(true));
      AndExp *pAndExp2 = new AndExp(new VariableExp('X'), new NotExp(new VariableExp('Y')));
      BooleanExp *pBExp = new OrExp(pAndExp1, pAndExp2);

      Context con;

// assign boolean values to variables in the expression
      con.assign('X', true);
      con.assign('Y', true);
      cout << pBExp->Evaluate(&con) << endl;
// assign other boolean values and re-evaluate
      con.assign('X', false);
      con.assign('Y', false);
      cout << pBExp->Evaluate(&con) << endl;
      delete pBExp;
}
```

The output of the program:

true
false

3. Command pattern

Command pattern is an object behavioral pattern that decouples invoker and receiver by encapsulating a request as an object. In this way, you can parameterize clients with different requests, queue or log requests, and support undo-able operations.

In the following example, Robot object is a receiver and RobotWalkCommand object （a derived Command object） has the right method (execute(), and actually, performing robot->walk()) of the receiver. Similarly, RobotTurnCommand object has the right method of performing robot->turn(). Drone object is another receiver. DroneFlyCommand has the right method of performing drone->fly(), and DroneLandCommand has the method of performing drone->land().

The derived command object has the pointer to the corresponding receiver as its member. For example, RobotWalkCommand object has the pointer to Robot object. As a result, when asked to perform an action on a receiver, we just feed the object to a ControlCenter obejct which is an invoker. The invoker makes a request of a Command object by calling that object's execute() method. Then, since the method knows what the receiver is, it invokes the action on that receiver.

The following main function is used to test the command pattern:

```
#include <iostream>
using namespace std;
// The client
int main() {
// Receiver
	Robot* robot = new Robot;
	Drone* drone = new Drone;
// concrete Command objects
	RobotWalkCommand* robotWalk = new RobotWalkCommand(robot);
	RobotTurnCommand* robotTurn = new RobotTurnCommand(robot);
	DroneFlyCommand* droneFly = new DroneFlyCommand(drone);
	DroneLandCommand* droneLand = new DroneLandCommand(drone);
// invoker objects
	ControlCenter* control = new ControlCenter;
// execute control->setCommand(robotWalk);
	control->buttonPressed();
	control->setCommand(robotTurn);
	control->buttonPressed();
	control->setCommand(droneFly);
	control->buttonPressed();
	control->setCommand(droneLand);
	control->buttonPressed();
	delete robot, drone, robotWalk, robotTurn, droneFly, droneLand, control;
	return 0;
}
```

The 8 classes involved in the main function are: a base class Command and its four derived classes RobotWalkCommand, RobotTurnCommand, DroneFlyCommand, DroneLandCommand; Robot and Drone; ControlCenter.
Here are example codes of Robot and DroneFlyCommand:

```
// Receiver Robot Class
class Robot {
public:
    void walk() { cout << "The robot is walking\n"; }
     void turn() { cout << "The robot is turning\n"; }
};
// Command for drone flying
class DroneFlyCommand : public Command {
public:
    DroneFlyCommand(Drone* drone) : m_drone(drone) {}
    void execute() { m_drone->fly(); }
private:
    Drone* m_drone;
};
```

Please implement the rest 6 classes such that the main function can produce the required output:

```
Button Pressed
The robot is walking
Button Pressed
The robot is turning
Button Pressed
The drone is flying
Button Pressed
The drone is landing
```

4. Observer Pattern Design

One-to-many relationship between objects is common in different application scenarios. For instance, if the information of one user in e-commerce website is changed, we need to notify other running objects dependent on the user information, such as account information display, purchase records, logistics scheduling, and so on. Observer pattern is frequently used to handle one-to-many relationship. In the development of user interface problem, observer pattern is used to handle model-view or data-display separation.

The basic design of the observer pattern is illustrated in the UML figure below:

In particular, the subject class should implement the interface functions to maintain an vector of observers and notify the observers if the value of the subject is changed. These interfaces are then shared across different types of concrete subjects. The observer class should define the interface that can be called by subject to calculate its output.

The following main function is used to test the observer pattern:

```
#include <iostream>
#include <vector>
using namespace std;
 int main() {
//a object to store a string
        ConcreteStringSubject s1;
//an observer to append a string "appended" to the subject and then print out the string
        ObserverAppend o1;
//an observer to judge if the string is “Manager” or not and then print true or false ObserverTitle o2;
        s1.addObserver(&o1);
        s1.addObserver(&o2);
        s1.setValue("Empolyee");
        s1.notify(); //should update all the observers
//a subject to store a int
        ConcreteIntSubject s2;
//an observer to increase the value by 10 and then print out the value
        ObserverIncrement o3;
//an observer to judge whether the value is less than 40 and then print the result is true //or false
        ObserverLess o4;
        s2.addObserver(&o3);
        s2.addObserver(&o4);
        s2.setValue(18);
        s2.notify(); //should update all the observer
        return 0;
}
```

The 8 classes involved in the main function are: two base classes Subject and Observer; ConcreteStringSubject and its two observers: ObserverAppend, ObserverTitle; ConcreteIntSubject and its two observers: ObserverIncrement, ObserverLess.

Requirement: the common interface functions of ConcreteStringSubject1 and ConcreteStringSubject2 that can be shared should be implemented in their parent class subject, so does the class observer.

Here are example codes of ConcreteStringSubject1 and ObserverLess:

```
class ConcreteStringSubject : public Subject {
public:
    ConcreteStringSubject() {}
    const string& getValue() { return m_str; }
    void setValue(const string& s) { m_str = s; }
private:
    string m_str;
};
```

```
class ObserverLess : public Observer {
public:
    ObserverLess() { }
    virtual void update(Subject *pSubject) {
        ConcreteIntSubject *pIntSubject = static_cast<ConcreteIntSubject
*>(pSubject);
        int v = pIntSubject->getValue();
        if (v < 40) cout << "True" << endl;
        else cout << "False" << endl;
    }
};
```

Please implement the rest 6 classes such that the main function can produce the required output:

```
Empolyee appended
False
28
True
```

5. design a circular queue template class

Design a generic circular queue class. using C++ standard exception class: overflow_error and underflow_error in the design.
the picture below shows the UML class diagram for the generic queue class.

CQueue<T>		
-queueSize:	const int	/*the queue size, is a const member, can store up to queueSize-1 datas*/
-head:	int	/*Record the subscript of the queue head*/
-rear:	int	/*Record the subscript of the end of the queue*/
-data_buff:	T*	/*Data storage buffer*/
+CQueue():		/*the default queue size is 10*/
+CQueue(int s)		
+~CQueue()		
+getSize():	int const	/*mean: the function return int , is a const member function*/
+getNumbers():	int const	/* Calculation formula for the total number of queue elements is : (rear - head + queueSize)%queueSize */
+getHead():	T	
+enQueue(value: T):	void	
+deQueue():	T	
+isEmpty():	bool const	
+isFull():	bool const	
+show():	void const	

The test code is in test.cpp, The ouput is (There is a space at the end of the line):
now the queue is full! 0 1 2 3 4 5 6 7 8 0 5 6 7 8

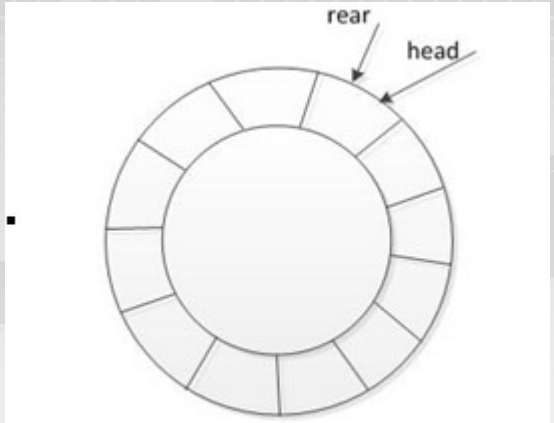


Figure 1 rear==head, The queue is empty

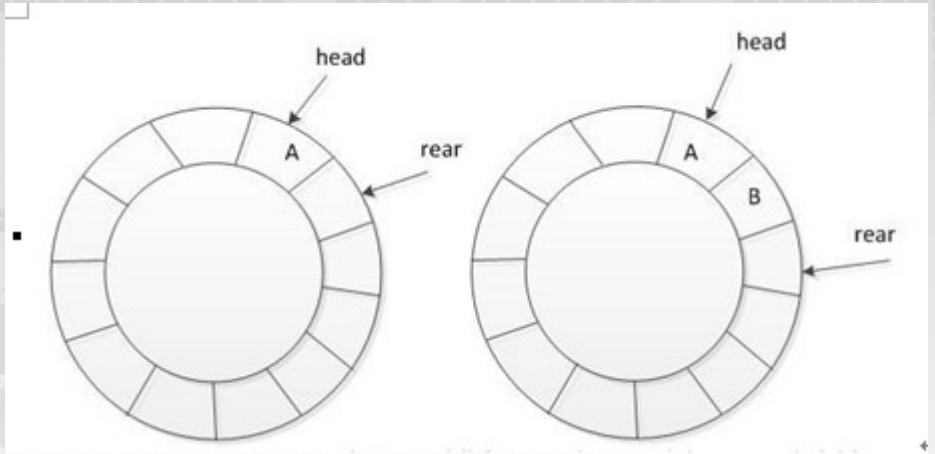


Figure 2 Add elements' A 'and' B 'to the queue in turn

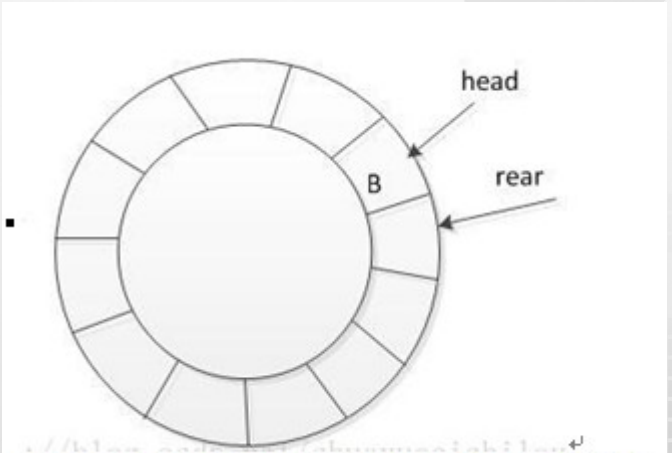


Figure 3 Put the element 'A' of the head out of the queue

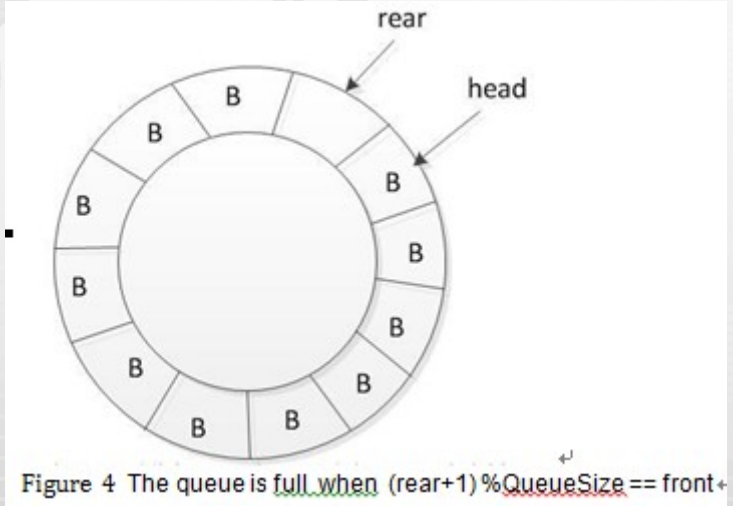
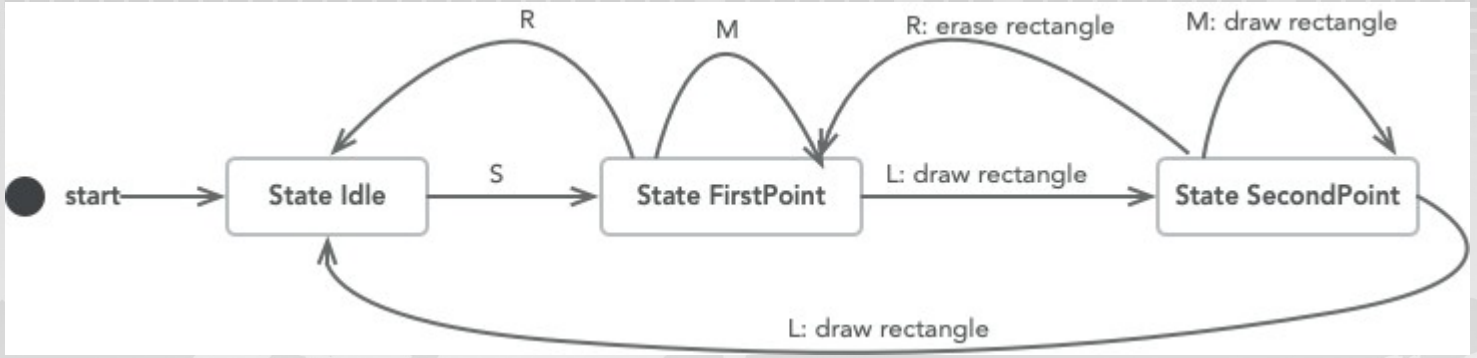


Figure 4 The queue is full when (rear+1) %QueueSize == front

```
/*test.cpp*/
#include <iostream>
#include <stdexcept>
using namespace std;
#include "CQueue.h"
int main() {
try {
	CQueue<double> rq;
	for (int i = 0; i < rq.getSize()-1; i++)
		rq.enQueue(i);
	if (rq.isFull())
		printf("now the queue is full! ");
	if (!rq.isEmpty())
		rq.show();
	cout << rq.getHead() << " ";
	for (int i = 0; i < 5; i++) // dequeueing 5 elements
		rq.deQueue();
	rq.show();
}
	catch (overflow_error& r) {
		cout << r.what(); }
	catch (underflow_error& r) {
		cout << r.what(); }
return 0;
}
```

The above description can be expressed as the following state diagram.



Each square in the diagram is a state, and the system starts from the Idle state. The capital letters above the lines indicate the received input. Next to the letter (followed by a colon), the text indicates the action to take when this input is received. The arrows indicate the state transfer when the input is received. The program reads an integer which is the number of the case. After that, the program reads a string from the standard input containing the sequence of "SMLR" characters and then performs the appropriate action based on this sequence. Once it enters a state, the program outputs a capital letter that represents the state. The letters of the three states are:

I for IDLE
F for FIRST
S for SECOND

The program also outputs the action to be performed, and there are only two actions to output:

D for DRAW

E for ERASE

The program ends when the input string ends, and does not need to output anything to indicate the end.

If an undefined input is received at one state, e.g., M received at IDLE , the program does nothing and keeps on the IDLE state.

You need to implement a class called state, and the subclasses of state for each state mentioned above. The state has a count variable to trace the number of objects instantiated by the derived classes of state.

There is another class StatePool which maintains all the possible objects of state . All the objects of State re created at the beginning of the program. And they can be retrieved by their names via get_state().

We provide some parts of State and StatePool for you. So do not forget to include them in your submission.

The following code is the main function of the program. From this code, you can derive all the necessary functions required for the State class. You must include all the provided lines from /* Your submit begins here*/ to /* Your submit ends here*/ in your submission.

Class Definition:

```
class State { };
class StatePool { };
```

Main Function:

```
#include <iostream>
#include <string>
 #include <map>
using namespace std;
/* Your submit begins here*/
class State {
public:
    static int get_count() { return count; }
protected:
    State() {count++;}
private:
    static int count; /* Feel free to add anthing needed.*/
};
```

```
class StatePool {
public:
    static void create_states();
    static void free_states() { /* put your code here*/ }
    static void set_state(string name, State* state) { mStates[name] = state; };
    static State* get_state(string name) { return mStates[name];
};
private:
    static map<string, State*> mStates;
};

/* Feel free to add anything needed here.*/
/* Find a suitable place to put the subclasses of State.*/
/* Your submit ends here*/
```

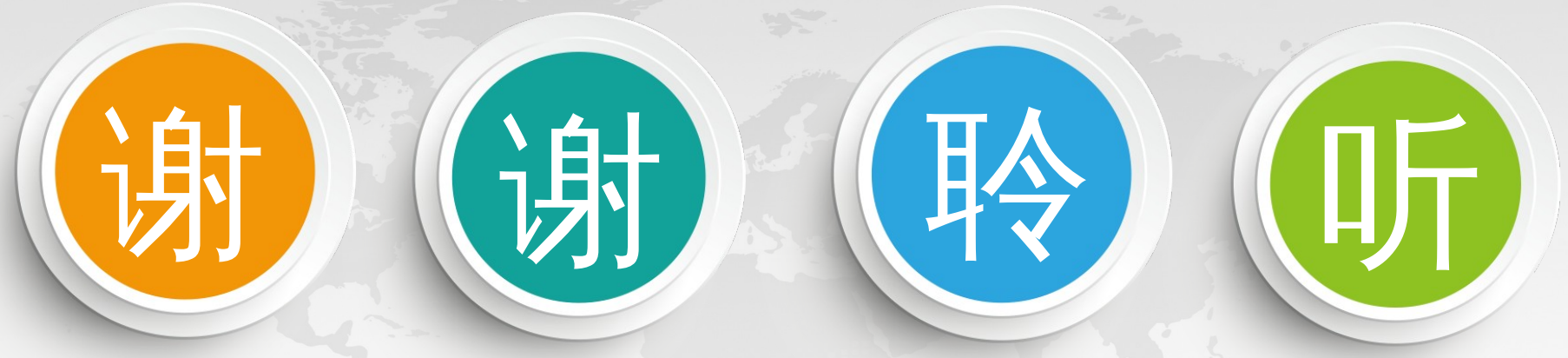
```
class StateTest : public State {
    virtual string toString() { return "T"; };
public:
    ~StateTest() { cout << "TEST"; }
 };

void StatePool::create_states() {
    set_state("IDLE", new StateIdle());
    set_state("FIRST", new StateFirst());
    set_state("SECOND", new StateSecond());
}
```

```
int main() {
	int case_num;
	cin >> case_num;
	cout << case_num;
	StatePool::create_states();
	string cmd;
	cin >> cmd;
	State *current_state = StatePool::get_state("IDLE");
	cout << current_state->toString();
	for ( int i=0; i<cmd.length(); i++ ) {
	// cout << cmd[i] << " ";
	switch (cmd[i]) {
		case 'S': current_state = current_state->cmdS(); break;
		case 'L': current_state = current_state->cmdL(); break;
		case 'R': current_state = current_state->cmdR(); break;
		case 'M': current_state = current_state->cmdM(); break;
		}
		cout << current_state->toString();
	}
	cout << endl;
	if (case_num==3) { StatePool::set_state("TEST", new StateTest()); }
	if (case_num>=2) {
		cout << State::get_count() << endl;
		StatePool::free_states();
		cout << State::get_count() << endl;
	}
}
```

Sample Input:
1LLSMMMLMML

Sample Output:
1IIIFFFFDSDSDSDI

代码长度限制	8k
时间限制	100 ms
内存限制	2 MB



THANKS!